

RELATÓRIO COMPLETO DE QUALIDADE E TESTE DE SOFTWARE

Projeto: Sistema de Agendamento para Barbearia

Disciplina: Qualidade e Teste de Software

Tecnologias: Flutter, Dart, ChangeNotifier (MVVM)

Normas de Referência: ISO/IEC/IEEE 29119-1, 29119-2 e 29119-4

Grupo 11

João Victor Avello

Julio Braidó

João Victor Viana

Ray Martins

1. Objetivo

Este documento apresenta o planejamento, especificação, implementação e resultados dos testes realizados no Sistema de Agendamento para Barbearia, desenvolvido em Flutter com arquitetura baseada em ChangeNotifier (MVVM). O objetivo é verificar se os requisitos funcionais de cadastro, login e agendamento foram implementados corretamente e se as regras de negócio presentes nos ViewModels estão funcionando conforme esperado.

2. Sistema Sob Teste

O sistema possui fluxo de autenticação (cadastro e login) e fluxo de agendamento de horários em uma barbearia. A camada de apresentação utiliza o padrão ChangeNotifier, concentrando as regras de negócio nos ViewModels. A comunicação com o backend é feita via HTTP (REST), sendo isolada nos testes por meio de MockClient.

Itens testados

- RegisterViewModel — responsável pelo cadastro de novos usuários
- LoginViewModel — responsável pela autenticação de usuários
- AuthService — camada de serviço de autenticação (login e sessão)
- RegisterService — camada de serviço de cadastro
- AgendamentoViewModel — responsável pelo fluxo completo de agendamento

Observação: O projeto não possui AuthRepositoryImpl. A camada equivalente é composta por AuthService e RegisterService, que realizam as chamadas HTTP diretamente ao backend Node.js.

3. Requisitos Funcionais

Módulo de Autenticação

Código	Descrição
RF01	O usuário deve conseguir se cadastrar.
RF02	O sistema deve impedir cadastro com campos vazios.
RF03	O sistema deve impedir cadastro com campo individual vazio (ex.: senha em branco).
RF04	O sistema deve impedir cadastro duplicado.
RF05	O sistema deve retornar true após cadastro bem-sucedido, sinalizando navegação para login.
RF06	O usuário deve conseguir realizar login.
RF07	O sistema deve impedir login com campos vazios.
RF08	O sistema deve impedir login inválido (credenciais incorretas).

Módulo de Agendamento

Código	Descrição
RF09	O sistema deve buscar e exibir os serviços disponíveis para seleção.
RF10	O sistema deve buscar e exibir as formas de pagamento disponíveis.
RF11	O sistema deve bloquear horários já ocupados, impedindo sua seleção.
RF12	O sistema deve impedir confirmação de agendamento sem horário selecionado.
RF13	O sistema deve impedir confirmação de agendamento sem serviço selecionado.
RF14	O sistema deve calcular o valor total dos serviços selecionados corretamente.
RF15	O sistema deve impedir agendamento de usuário não autenticado.

4. Estratégia de Testes

Foram aplicados testes de unidade sobre os ViewModels e Services utilizando MockClient (package:http/testing.dart) para isolamento das dependências HTTP, e SharedPreferences.setMockInitialValues para simulação da sessão do usuário. Os testes verificam valores de retorno dos métodos, mensagens de erro expostas (errorMessage) e estados internos (isLoading, submitting, selectedTime, total).

Técnicas Utilizadas

- Particionamento de Equivalência — classes válidas e inválidas de entrada
- Análise de Valor Limite — strings vazias, listas vazias, campos null
- Transição de Estado — autenticado/não autenticado, horário disponível/bloqueado
- Teste Baseado em Cenário — fluxo principal e fluxos alternativos do diagrama de sequência

5. Casos de Teste

5.1 Módulo de Autenticação — RegisterViewModel

TC	Nome	Entrada	Resultado Esperado
TC01	Cadastro com dados válidos	Nome, nomeUsuário, e-mail e senha válidos; API retorna 201	register() retorna true; errorMessage é null
TC02	Cadastro com todos os campos vazios	Todos os campos em branco	register() retorna false; errorMessage = 'Preencha todos os campos!'
TC03	Cadastro com campo individual vazio	Senha em branco; demais preenchidos	register() retorna false; errorMessage = 'Preencha todos os campos!'
TC04	Cadastro duplicado	E-mail já cadastrado; API retorna 409	register() retorna false; errorMessage contém mensagem de erro

TC05	Sinalização de retorno ao login	Cadastro bem-sucedido; API retorna 201	register() retorna true; isLoading = false; errorMessage é null
-------------	--	--	---

5.2 Módulo de Autenticação — LoginViewModel

TC	Nome	Entrada	Resultado Esperado
TC06	Login válido	Username e password corretos; API retorna user válido	login() retorna true; errorMessage é null
TC07	Login com campos vazios	Username e password em branco	login() retorna false; errorMessage = 'Preencha todos os campos!'
TC08	Login inválido	Username correto + password errado; API retorna user null	login() retorna false; errorMessage = 'Credenciais inválidas!'
TC09	Estado após login bem-sucedido	Credenciais corretas; API retorna user válido	login() retorna true; isLoading = false; errorMessage é null

5.3 Módulo de Autenticação — AuthService e RegisterService

TC	Nome	Entrada	Resultado Esperado
TC01	register() com dados válidos	Dados válidos; API 201 com RegisterResponseModel	Retorna RegisterResponseModel preenchido; nome e e-mail corretos
TC04	register() com e-mail duplicado	E-mail existente; API 409	Lança Exception
TC06	login() com credenciais válidas	Credenciais corretas; API retorna user	LoginResponseModel com user != null; usuário salvo no SharedPreferences
TC07	login() com campos vazios	Campos vazios; API retorna user null	LoginResponseModel com user == null; message = Credenciais inválidas!
TC08	login() com senha incorreta	Senha errada; API 401	LoginResponseModel com user == null; getLoggedUser() retorna null
TC--	logout() remove sessão local	Usuário logado; chamada a logout()	getLoggedUser() retorna null após logout

5.4 Módulo de Agendamento — AgendamentoViewModel

TC	Nome	Entrada	Resultado Esperado
TC01	init() carrega dados corretamente	API retorna serviços, pagamentos e usuário logado	services e paymentMethods preenchidos; isLoading = false; loggedUserId = 1
TC02	confirmar() com dados válidos	Horário, serviço e pagamento selecionados; API aceita POST	confirmar() retorna null; submitting = false

TC03	confirmar() sem horário selecionado	selectedTime = null; serviço e pagamento selecionados	Retorna 'Selecione um horário.'
TC04	confirmar() sem serviço selecionado	Horário e pagamento selecionados; nenhum serviço	Retorna 'Selecione pelo menos um serviço.'
TC05	selectTime() bloqueia horário indisponível (A1)	API retorna 09:00 como ocupado; tentativa de seleção	selectedTime permanece null
TC06	selectDate() recarrega horários	Nova data selecionada; API retorna lista vazia	selectedDate atualizado; selectedTime = null
TC07	total calculado com múltiplos serviços	Corte (R\$30) + Barba (R\$20) selecionados	total = 50.0; selectedServices.length = 2
TC08	confirmar() sem usuário autenticado	SharedPreferences vazio; sem sessão ativa	Retorna 'Usuário não identificado. Faça login novamente.'

6. Rastreabilidade

6.1 Autenticação

RF	TC Vinculado	Descrição
RF01	TC01 (RegisterViewModel)	Cadastro com dados válidos
RF02	TC02 (RegisterViewModel)	Cadastro com todos os campos vazios
RF03	TC03 (RegisterViewModel)	Cadastro com campo individual vazio
RF04	TC04 (RegisterViewModel)	Cadastro duplicado
RF05	TC05 (RegisterViewModel)	Sinalização de retorno ao login
RF06	TC06, TC09 (LoginViewModel)	Login válido e estado pós-login
RF07	TC07 (LoginViewModel)	Login com campos vazios
RF08	TC08 (LoginViewModel)	Login inválido

6.2 Agendamento

RF	TC Vinculado	Descrição
RF09	TC01 (AgendamentoViewModel)	Carregamento de serviços disponíveis
RF10	TC01 (AgendamentoViewModel)	Carregamento de formas de pagamento
RF11	TC05 (AgendamentoViewModel)	Bloqueio de horário indisponível (fluxo A1)
RF12	TC03 (AgendamentoViewModel)	Confirmação sem horário selecionado

RF13	TC04 (AgendamentoViewModel)	Confirmação sem serviço selecionado
RF14	TC07 (AgendamentoViewModel)	Cálculo do valor total dos serviços
RF15	TC08 (AgendamentoViewModel)	Bloqueio de agendamento sem usuário autenticado

7. Ambiente de Testes

- Flutter SDK
- Dart SDK
- flutter_test — framework principal de testes unitários
- package:http/testing.dart (MockClient) — interceptação de chamadas HTTP sem servidor real
- SharedPreferences.setMockInitialValues — simulação do armazenamento local de sessão
- Arquitetura ChangeNotifier (MVVM)
- Backend Node.js + Express.js + MySQL (isolado nos testes via MockClient)

Observação: FakeAuthService não existe neste projeto. O isolamento de dependências é realizado via injeção de MockClient no construtor dos Services, permitindo simular qualquer resposta HTTP sem necessidade de servidor ativo.

8. Código Avaliado

RegisterViewModel

Método register() — retorna bool. Valida campos vazios antes de chamar o RegisterService. Em caso de erro de rede ou rejeição da API, atualiza errorMessage com a mensagem recebida. Expõe isLoading para controle de estado da UI.

LoginViewModel

Método login() — retorna bool. Valida campos vazios antes de chamar o AuthService. Em caso de credenciais inválidas (user == null na resposta), atualiza errorMessage. Expõe isLoading para controle de estado da UI. A navegação para Home é responsabilidade da LoginPage, que interpreta o bool retornado.

AuthService e RegisterService

Camada de serviço que realiza chamadas HTTP ao backend. Testada de forma isolada via MockClient. AuthService.login() retorna LoginResponseModel e persiste o usuário no SharedPreferences quando bem-sucedido. AuthService.logout() remove a sessão local. RegisterService.register() retorna RegisterResponseModel em caso de sucesso.

AgendamentoViewModel

Método `init()` — carrega serviços, pagamentos, usuário logado e horários indisponíveis em paralelo via `Future.wait`. Método `confirmar()` — retorna `String?` (null = sucesso, mensagem = erro). Valida horário, serviço e pagamento selecionados, além da presença de usuário autenticado. Método `selectTime()` — bloqueia seleção de horários indisponíveis. Getter `total` — calcula soma dos valores dos serviços selecionados.

9. Resultados Obtidos

Todos os casos de teste foram executados com sucesso após os ajustes de arquitetura. Nenhuma falha foi identificada nas regras de negócio implementadas. Os requisitos funcionais definidos para autenticação (RF01–RF08) e agendamento (RF09–RF15) foram atendidos integralmente.

Módulo	TCs	Total	Passou	Falhou	Status
RegisterViewModel	TC01 ao TC05	5	5	0	✓ Aprovado
LoginViewModel	TC06 ao TC09	4	4	0	✓ Aprovado
AuthService	TC06, TC07, TC08, TC--	4	4	0	✓ Aprovado
RegisterService	TC01, TC04	2	2	0	✓ Aprovado
AgendamentoViewModel	TC01 ao TC08	8	8	0	✓ Aprovado

10. Conclusão

Com base nos testes automatizados implementados para `RegisterViewModel`, `LoginViewModel`, `AuthService`, `RegisterService` e `AgendamentoViewModel`, conclui-se que o sistema apresenta comportamento consistente e aderente aos requisitos. As validações de entrada, prevenção de erros, controle de estado, cálculo de totais e bloqueio de horários indisponíveis foram verificados e aprovados.

O projeto demonstra boa separação de responsabilidades: os ViewModels concentram as regras de negócio e expõem estado simples (`bool`, `String?`, `double`), enquanto os Services encapsulam a comunicação HTTP. Essa estrutura facilitou a testabilidade via injeção de `MockClient`, eliminando a necessidade de um servidor ativo durante os testes.

A cobertura foi expandida em relação à versão anterior do relatório para incluir o módulo de agendamento (RF09–RF15), alinhando a documentação ao código efetivamente produzido e testado pelo grupo.